PDF-to-HTML Conversion Evaluation Framework

Prepared for: Salesforce
Prepared by: Amazon Web Services — danamord@amazon.com Sr. Technical Account Management
Date: March 2026
Classification: Confidential

Executive Overview

Your team processes large volumes of unstructured PDF documents through a DocLing-based conversion pipeline to produce structured HTML. Evaluating the quality of these conversions has been a persistent challenge: without Ground Truth datasets, there was no systematic way to identify where conversions fail or to measure improvement over time.

We developed a comprehensive evaluation framework that solves this problem without requiring Ground Truth. The framework combines four quantitative metrics with thirteen diagnostic tests, each grounded in real production failures observed across your document corpus. This gives your team an actionable, automated way to score every conversion, identify root causes of failure, and prioritize pipeline improvements.

> **Key Result**
>
> The framework identified a single bug (KaTeX parser misinterpreting $ characters) that, once fixed, is projected to improve conversion scores by 10–15 points across all affected documents — the highest-ROI fix in the pipeline.

What We Built

Two-Layer Evaluation Architecture

The framework uses two complementary layers, like a medical checkup that starts with vital signs before running specific diagnostic tests.

Layer 1: Quantitative Metrics (Automated Vital Signs)

Four metrics computed per page, providing fast, automated, reproducible scoring:

Metric	Measures	Example
CER	Character accuracy	CER of 36 = ~36% of characters differ
WER	Word accuracy	WER of 100 = nearly all words wrong or missing
Sim	Content retention	Sim of 82 = 82% of source text preserved
SSIM	Visual fidelity	SSIM of 36 = layout significantly different

Layer 2: 13 Diagnostic Tests (Root Cause Analysis)

When metrics flag a problem, the diagnostic tests pinpoint why. Each test targets a specific failure mode observed in real production data across your 7 reference documents:

#	Test	Severity	What It Catches
1	Merged Cell Preservation	Critical	Table headers duplicated instead of spanning columns
2	Table Structure Detection	Critical	Tables rendered as bullet lists
3	Images in Tables	High	Images ripped out of table cells

4	Header/Footer Contamination	High	Page headers injected into body text
5	TOC Alignment	Medium	Table of contents entries missing or misaligned
6	Cover Page Layout	Medium	First page distorted or images missing
7	Content Completeness	Critical	Entire pages or sections silently missing
8	Formula Preservation	Medium	Equations flattened to unreadable text
9	Warning Block Preservation	High	Warning boxes corrupted by parser errors
10	Icon Inline Rendering	Medium	Icons replaced by wrong characters
11	List Structure	Medium	Nested lists flattened to flat bullets
12	Text Parsing Accuracy	Critical	Parser errors corrupt readable text (KaTeX bug)
13	Element Positioning	Low	Elements relocated to wrong positions

Key Findings

We evaluated 7 production PDF documents spanning technical manuals, marketing fact sheets, safety documents, and service bulletins. Scores ranged from Critical (44) to Good (88):

Document	Total	CER	WER	Sim	SSIM	Quality
LPA Omega Service Bulletin	87.69	3.14	4.29	97.16	69.91	Good
Acura RDX Fact Sheet	74.16	12.71	52.17	81.98	76.89	Moderate
CAREL ir33 (cover)	62.34	36.00	77.84	82.38	65.75	Poor
CAREL ir33 (page 1)	44.06	53.43	99.80	77.79	35.66	Critical

Top Failure Patterns Identified

1. **KaTeX Parser Bug (Tests 9, 12)** — The rendering layer misinterprets `$` characters in internal references as LaTeX math delimiters, corrupting entire paragraphs. Affects multiple documents. This is a one-line fix with the highest ROI.
2. **Table Structure Failures (Tests 1, 2, 3)** — Merged cells get duplicated instead of spanning, tables render as bullet lists, and images are ripped out of table cells.
3. **Content Loss (Tests 5, 7)** — Entire pages and TOC entries silently disappear. One document lost 8 sections from its table of contents with no indication of failure.
4. **Layout Contamination (Tests 4, 6, 13)** — Page headers/footers inject into body text, cover pages are distorted, and elements relocate to wrong positions.

How to Use the Framework

The framework supports multiple evaluation strategies that can be adopted incrementally:

Approach	How It Works	Best For
Heuristic Scripts	Automated checks that detect parse errors, duplicate text, orphaned images, and div artifacts	CI/CD integration; runs in < 1 second per document
LLM-as-Judge	Multimodal AI compares rendered PDF pages against HTML screenshots using structured evaluation prompts	Detailed diagnosis without Ground Truth; scales to hundreds of documents
Hybrid (Recommended)	Heuristics on every conversion + LLM-as-judge on flagged pages + random sampling of passing pages	Production deployment; balances cost, speed, and coverage

Recommended Next Steps

Phase 1: Quick Wins (1–2 weeks)

- **Fix the KaTeX parser bug** — a single pipeline fix that eliminates an entire class of errors across all documents
- **Baseline all 7 reference documents** — run through the Pipeline Test Coordinator to establish benchmark scores
- **Implement top 3 heuristic checks** — ParseError detector, duplicate header detector, and content completeness ratio

Phase 2: Automated Evaluation Pipeline (2–4 weeks)

- **Build the full 13-test heuristic suite** as a Python package with CLI interface and JSON output
- **Integrate LLM-as-judge** using Claude via Bedrock for multimodal visual comparison of PDF vs HTML pages
- **Connect to Pipeline Test Coordinator** for real-time scoring visibility and filtering by failure type

Phase 3: Scale & Improve (4–8 weeks)

- **Expand test corpus** to 30+ diverse PDFs covering academic papers, financial reports, government forms, and scanned documents
- **Prioritize DocLing pipeline fixes** based on diagnostic data: merged cell detection, header/footer filtering, table vs list classification
- **Explore alternative extraction tools** — Amazon Textract for tables/forms, olmOCR for cross-validation, Bedrock Nova VLM for vision-based conversion

Phase 4: Production Quality Gates (8–12 weeks)

- **CI/CD integration** — every conversion triggers heuristics (< 1s), metrics (< 5s), and LLM-as-judge on flagged pages (< 30s)
- **Monitoring dashboard** tracking scores over time, alerting on regressions, and comparing pipeline alternatives

Target Outcomes

Metric	Current	Phase 1	Phase 4
Mean Total Score	~60	70	85+
Pages with ParseErrors	~15%	0%	0%
Evaluation coverage	7 documents	7 docs, all pages	100+ documents
Time to evaluate	Manual (hours)	5 minutes	< 1 minute
Automation level	0%	50%	95%

The complete evaluation framework is provided in the accompanying documentation package, described below.

Accompanying Materials

This executive summary is part of a deliverable package containing the full technical framework and supporting evidence:

File	Description
`SKILL.md`	Complete evaluation framework. Contains all 13 test definitions with observed failure examples, scoring rubrics (Pass/Partial/Fail), heuristic check code templates in Python, LLM-as-judge evaluation prompts, metric interpretation benchmarks, and CI/CD integration patterns. **This is the primary technical reference.**
`GUIDE.md`	Explanation guide for stakeholders and new team members. Covers how the two evaluation layers work together, how to run an evaluation (manual, LLM-as-judge, heuristic, or hybrid), and how to interpret score reports with a real worked example.
`examples/`	Screenshot evidence organized by document and test number. Includes side-by-side Original PDF vs HTML Output comparisons for all 7 reference documents, plus Pipeline Test Coordinator screenshots showing scores across the quality spectrum (Total 44 – 88). See `examples/INDEX.md` for a full catalog.

We look forward to partnering with your team to implement these improvements and drive measurable gains in conversion quality.